보상 함수 시스템 분석

1. 보상 함수 시스템 개요

Open R1의 보상 함수 시스템은 GRPO 훈련에서 모델의 응답 품질을 평가하는 핵심 컴포넌트입니다. 다양한 측면에서 모델의 성능을 측정하여 강화학습 과정을 안내합니다.

1.1 시스템 구조

```
# 보상 함수 등록 시스템

REWARD_FUNCS_REGISTRY = {

"accuracy": accuracy_reward, # 정답 정확도

"format": format_reward, # 형식 준수

"reasoning_steps": reasoning_steps_reward, # 추론 단계

"code": code_reward, # 코드 실행

"length": len_reward, # 길이 기반

# ... 더 많은 보상 함수들

}
```

1.2 멀티모달 보상 시스템

Open R1은 여러 보상 함수를 동시에 사용하여 다면적 평가를 수행합니다:

- **수학 문제**: accuracy + format + tag_count
- **코딩 문제**: code + code_format + reasoning_steps

- **일반 추론**: format + reasoning_steps + repetition_penalty

2. 핵심 보상 함수들

2.1 정확도 보상 (accuracy_reward)

수학 문제의 정답 여부를 검증하는 가장 기본적인 보상 함수입니다.

```
def accuracy_reward(completions, solution, **kwargs):
"""정답과 모델 응답을 비교하여 이진 보상 제공"""
contents = [completion[0]["content"] for completion in completions]
rewards = []
for content, sol in zip(contents, solution):
# 1. 정답을 LaTeX 형식으로 파싱
gold_parsed = parse(sol, extraction_mode="first_match")
if len(gold_parsed) != 0:
# 2. 모델 응답에서 답안 추출
answer_parsed = parse(
content,
extraction_config=[LatexExtractionConfig(
normalization_config=NormalizationConfig(
nits=False,
malformed_operators=False,
basic_latex=True,
equations=True,
```

```
boxed="all",
units=True,
),
boxed_match_priority=0,
try_extract_without_anchor=False,
)]
)
# 3. 정답 검증
try:
reward = float(verify(gold_parsed, answer_parsed))
except Exception as e:
reward = None # 검증 실패 시 예시 스킵
else:
reward = None # 파싱 불가능한 정답은 스킵
rewards.append(reward)
return rewards
```

주요 특징

- **LaTeX 파싱**: math_verify 라이브러리 사용
- **정규화**: 수식 표현의 다양한 형태를 표준화
- **Boxed 우선순위**: `\boxed{}` 형태의 답안을 우선 추출
- **오류 처리**: 파싱/검증 실패 시 해당 예시 스킵

2.2 형식 보상 (format_reward)

DeepSeek-R1의 특별한 추론 형식을 준수하는지 검사합니다.

```
def format_reward(completions, **kwargs):
"""<think>...</think>와 <answer>...</answer> 형식 검증"""
pattern = r"^<think>\n.*?\n</think>\n<answer>\n.*?\n</answer>$"
completion_contents = [completion[0]["content"] for completion in completions]
matches = [re.match(pattern, content, re.DOTALL | re.MULTILINE)
for content in completion_contents]
return [1.0 if match else 0.0 for match in matches]
```

요구 형식

```
<think>
단계별 추론 과정이 여기에 들어갑니다.
문제를 분석하고, 해결 방법을 찾고,
중간 계산을 수행합니다.
</think>
<answer>
최종 답변이 여기에 들어갑니다.
</answer>
```

2.3 태그 개수 보상 (tag_count_reward)

형식 보상의 세밀한 버전으로, 각 태그의 정확한 개수를 검사합니다.

```
def tag_count_reward(completions, **kwargs):
"""각 태그가 정확히 한 번씩 나타나는지 검사"""
def count_tags(text: str) -> float:
count = 0.0
if text.count("<think>\n") == 1: # 시작 태그
count += 0.25
if text.count("\n</think>\n") == 1: # 종료 태그
count += 0.25
if text.count("\n<answer>\n") == 1: # 답변 시작
count += 0.25
if text.count("\n</answer>") == 1: # 답변 종료
count += 0.25
return count
contents = [completion[0]["content"] for completion in completions]
return [count_tags(c) for c in contents]
```

점수 체계

- 각 태그당 0.25점 (총 1.0점 만점)
- 태그가 누락되거나 중복되면 해당 점수 차감
- 부분 점수 허용으로 점진적 학습 지원

2.4 추론 단계 보상 (reasoning_steps_reward)

명확한 단계별 추론을 장려하는 보상 함수입니다.

```
def reasoning_steps_reward(completions, **kwargs):
"""단계별 추론 패턴 검출"""
pattern = r"(Step \d+:|^\d+\.|\n-|\n\*|First,|Second,|Next,|Finally,)"
completion_contents = [completion[0]["content"] for completion in completions]
matches = [len(re.findall(pattern, content)) for content in completion_contents]
# 3단계 이상을 권장, 그 이하는 부분 점수
return [min(1.0, count / 3) for count in matches]
```

인식 패턴

- `Step 1:`, `Step 2:` 등의 명시적 단계
- `1.`, `2.` 등의 번호 매기기
- `-`, `*` 등의 불릿 포인트
- `First,`, `Next,`, `Finally,` 등의 전환어

3. 길이 기반 보상 함수들

3.1 길이 보상 (len_reward)

Kimi 1.5 기술 보고서에서 영감을 받은 길이 기반 보상입니다.

```
def len_reward(completions, solution, **kwargs):
"""길이 기반 보상으로 과도한 사고 억제"""
contents = [completion[0]["content"] for completion in completions]
```

```
# 1. 정답 여부 확인
correctness = [verify_answer(content, sol)
for content, sol in  zip(contents, solution)]
# 2. 길이 계산
lengths = [len(content) for content in contents]
min_len, max_len = min(lengths), max(lengths)
if max_len == min_len:
return [0.0] * len(completions)
# 3. 길이 기반 보상 계산
rewards = []
for length, is_correct in  zip(lengths, correctness):
lambda_val = 0.5 - (length - min_len) / (max_len - min_len)
if is_correct:
reward = lambda_val # 정답: 짧을수록 높은 보상
else:
reward = min(0, lambda_val) # 오답: 음수 보상만
rewards.append(float(reward))
return rewards
```

보상 체계

- **정답인 경우**: 짧은 응답일수록 높은 보상
- **오답인 경우**: 긴 응답일수록 더 큰 페널티
- **토큰 효율성**: 불필요한 장황함 억제

3.2 코사인 스케일 보상 (get_cosine_scaled_reward)

길이에 따른 코사인 스케줄링을 적용한 보상 함수입니다.

```
def get_cosine_scaled_reward(
min_value_wrong=-1.0,
max_value_wrong=-0.5,
min_value_correct=0.5,
max_value_correct=1.0,
max_len=1000
):
def cosine_scaled_reward(completions, solution, **kwargs):
"""코사인 스케줄링으로 길이 기반 보상 조정"""
for content, sol in zip(contents, solution):
is_correct = verify_answer(content, sol)
gen_len = len(content)
# 코사인 스케일링 적용
progress = gen_len / max_len
cosine = math.cos(progress * math.pi)
if is_correct:
min_val, max_val = min_value_correct, max_value_correct
else:
min_val, max_val = max_value_wrong, min_value_wrong
reward = min_val + 0.5 * (max_val - min_val) * (1.0 + cosine)
```

```
rewards.append(float(reward))

return rewards

return cosine_scaled_reward
```

특징

- **부드러운 스케일링**: 선형 대신 코사인 함수 사용
- **매개변수화**: 정답/오답별 보상 범위 조정 가능
- **길이 적응적**: 최대 길이 기준으로 스케일링

3.3 반복 페널티 보상 (get_repetition_penalty_reward)

N-gram 반복을 감지하여 페널티를 부과합니다.

```
def  get_repetition_penalty_reward(ngram_size=3, max_penalty=-1.0, language="en"

def  repetition_penalty_reward(completions, **kwargs):

"""N-gram 반복 페널티"""

contents = [completion[0]["content"] for completion in completions]

rewards = []

for completion in contents:

if completion == "":

rewards.append(0.0)

continue

# N-gram 생성 및 중복 검사

ngrams = set()

total = 0
```

```
ngram_array, words = zipngram(completion, ngram_size)

for ng in ngram_array:

ngrams.add(ng)

total += 1

# 중복도 계산

scaling = 1 - len(ngrams) / total

reward = scaling * max_penalty

rewards.append(reward)

return rewards

return repetition_penalty_reward
```

언어별 지원

- **영어**: 공백 기준 단어 분할
- **중국어**: jieba 라이브러리 사용
- **확장 가능**: 다른 언어 추가 가능

4. 코드 실행 보상 함수들

4.1 기본 코드 보상 (code_reward)

코드 스니펫을 실제 실행하여 테스트 케이스 통과율을 측정합니다.

```
def  code_reward(completions, num_parallel=2, provider_type="e2b", **kwargs):

"""코드 실행 기반 보상"""

# 1. 코드 추출
```

```
code_snippets = [extract_code(completion[-1]["content"])

for completion in completions]

# 2. 실행 스크립트 생성

evaluation_script_template = """

import subprocess

import json

def evaluate_code(code, test_cases):

passed = 0

total = len(test_cases)

for case in test_cases:

process = subprocess.run(

["python3", "-c", code],

input=case["input"],

text=True,

capture_output=True,

timeout=5

)

if process.returncode == 0:

output = process.stdout.strip()

if output == case["output"].strip():

passed += 1

return passed / total

code_snippet = {code}
```

```
test_cases = json.loads({test_cases})

evaluate_code(code_snippet, test_cases)

"""

# 3. 실행 제공자를 통한 평가

execution_provider = get_provider(

provider_type=provider_type,

num_parallel=num_parallel,

**kwargs

)

return execution_provider.execute_scripts(scripts, ["python"] * len(scripts))
```

지원 제공자

- **E2B**: Python 특화 클라우드 샌드박스
- **Morph**: 다중 언어 지원 샌드박스
- **Local**: 로컬 실행 환경

4.2 이진 코드 보상 (binary_code_reward)

코드 보상을 이진 값으로 변환합니다.

```
def  binary_code_reward(completions, **kwargs):

"""코드 보상을 0/1로 이진화"""

rewards = code_reward(completions, **kwargs)

BINARY_THRESHOLD = 0.99
```

```
output = []
for reward in rewards:
if reward is None:
output.append(None)
else:
output.append(1.0  if reward > BINARY_THRESHOLD else  0.0)
return output
```

4.3 경쟁 프로그래밍 보상

IOI 보상 (ioi_code_reward)

```
def  ioi_code_reward(completions, test_batch_size=1, provider_type="piston", **k
"""IOI 문제 평가"""
# C++ 코드 추출 및 헤더 추가
code_snippets = [
add_includes(extract_code(completion[-1]["content"], "cpp"), problem_id)
for completion, problem_id in  zip(completions, kwargs["id"])
]
# 비동기 평가 실행
loop = _init_event_loop()
evals = [
loop.create_task(score_subtask(client, problem_data, code, test_batch_size))
for problem_data, code in  zip(problems_data, code_snippets)
```

```
]
results = loop.run_until_complete(asyncio.gather(*evals))
return [result.score for result in results]
```

CodeForces 보상 (cf_code_reward)

```
def cf_code_reward(completions, scoring_mode="weighted_sum", **kwargs):
"""CodeForces 문제 평가"""
# 언어별 코드 추출
languages = kwargs.get("language", [None] * len(completions))
code_snippets = [
extract_code(completion[-1]["content"], language)
for completion, language in zip(completions, languages)
]
# 점수 계산 모드
# - pass_fail: 모든 테스트 통과 시에만 1점
# - partial: 통과한 테스트 비율
# - weighted_sum: 가중합 점수
return results
```

4.4 코드 형식 보상 (get_code_format_reward)

코딩 문제에 특화된 형식 보상입니다.

```
def get_code_format_reward(language="python"):
```

```
def code_format_reward(completions, **kwargs):
"""코드 블록이 포함된 형식 검증"""
languages = kwargs.get("language", [language] * len(completions))
completion_contents = [completion[0]["content"] for completion in completions]
matches = [
re.match(
rf"^<think>\n.*?\n</think>\n<answer>\n.*?```{lang}.*?```.*?\n</answer>$",
content,
re.DOTALL | re.MULTILINE
)
for content, lang in zip(completion_contents, languages)
]
return [1.0 if match else 0.0 for match in matches]
return code_format_reward
```

5. 고급 보상 함수들

5.1 과도한 길이 페널티 (get_soft_overlong_punishment)

DAPO 논문에서 제안된 부드러운 길이 페널티입니다.

```
def get_soft_overlong_punishment(max_completion_len, soft_punish_cache):
def soft_overlong_punishment_reward(completion_ids, **kwargs):
"""토큰 ID 기반 길이 페널티"""
```

```
rewards = []

for ids in completion_ids:

completion_length = len(ids)

if completion_length <= max_completion_len - soft_punish_cache:

rewards.append(0.0) # 페널티 없음

elif completion_length <= max_completion_len:

# 부드러운 페널티 적용

penalty = (max_completion_len - soft_punish_cache - completion_length) / soft_pu

rewards.append(penalty)

else:

rewards.append(-1.0) # 최대 페널티

return rewards

return soft_overlong_punishment_reward
```

페널티 구간

- **안전 구간**: `[0, max_len - cache]` → 페널티 없음
- **경고 구간**: `(max_len - cache, max_len]` → 선형 페널티
- **위험 구간**: `(max_len, ∞)` → 최대 페널티

6. 보상 함수 조합 및 가중치

6.1 보상 함수 등록 시스템

```
def  get_reward_funcs(script_args):
```

```
"""설정에 따른 보상 함수 조합"""

REWARD_FUNCS_REGISTRY = {

"accuracy": accuracy_reward,

"format": format_reward,

"reasoning_steps": reasoning_steps_reward,

"cosine": get_cosine_scaled_reward(...),

"repetition_penalty": get_repetition_penalty_reward(...),

"length": len_reward,

"code": partial(code_reward, ...),

"binary_code": partial(binary_code_reward, ...),

"ioi_code": partial(ioi_code_reward, ...),

"cf_code": partial(cf_code_reward, ...),

"code_format": get_code_format_reward(...),

"tag_count": tag_count_reward,

"soft_overlong_punishment": get_soft_overlong_punishment(...),

}

# 설정된 보상 함수들만 선택

reward_funcs = [REWARD_FUNCS_REGISTRY[func] for func in script_args.reward_funcs

return reward_funcs
```

6.2 일반적인 조합 패턴

수학 문제

```
reward_funcs:
- accuracy  # 정답 여부 (주요)
- format  # 형식 준수
- tag_count  # 태그 정확성
```

코딩 문제

```
reward_funcs:
- code  # 코드 실행 (주요)
- code_format  # 코드 형식
- reasoning_steps  # 추론 과정
```

일반 추론

```
reward_funcs:
- format  # 기본 형식
- reasoning_steps  # 추론 단계
- repetition_penalty  # 반복 억제
```

7. 성능 최적화

7.1 비동기 보상 계산

```
async def compute_rewards_async(completions, reward_funcs):
```

```
"""여러 보상 함수를 병렬로 실행"""
tasks = []
for func_name, func in reward_funcs.items():
task = asyncio.create_task(func(completions))
tasks.append(task)
results = await asyncio.gather(*tasks)
return dict(zip(reward_funcs.keys(), results))
```

7.2 캐싱 및 배치 처리

```
# 코드 실행 결과 캐싱
@lru_cache(maxsize=1000)
def cached_code_execution(code_hash, test_cases_hash):
return execute_code(code, test_cases)

# 배치 단위 평가
def batch_evaluate(completions, batch_size=32):
for i in range(0, len(completions), batch_size):
batch = completions[i:i+batch_size]
yield evaluate_batch(batch)
```

7.3 오류 처리 및 복구

```python
def safe_reward_func(completions, **kwargs):
"""안전한 보상 함수 래퍼"""
try:
return compute_reward(completions, **kwargs)
except Exception as e:
logger.warning(f"Reward computation failed: {e}")
return [None] * len(completions) # 실패 시 스킵
```

8. 실제 사용 예시

8.1 YAML 설정 예시

```yaml
# 수학 문제용 설정
reward_funcs:
- accuracy
- format
- tag_count

# 고급 설정 (가중치 포함)
reward_funcs:
accuracy:
weight: 0.7
format:
```

```yaml
weight: 0.2
tag_count:
weight: 0.1

# 코사인 스케일링 매개변수
cosine_min_value_wrong: -1.0
cosine_max_value_wrong: -0.5
cosine_min_value_correct: 0.5
cosine_max_value_correct: 1.0
cosine_max_len: 1000

# 반복 페널티 설정
repetition_n_grams: 3
repetition_max_penalty: -1.0

# 코드 실행 설정
code_provider: e2b
parallel_code_exec_per_proc: 2
code_eval_test_batch_size: 1
code_eval_scoring_mode: weighted_sum
```

8.2 커스텀 보상 함수 추가

```python
def  custom_reward(completions, **kwargs):
"""사용자 정의 보상 함수"""
# 커스텀 로직 구현
rewards = []
for completion in completions:
content = completion[0]["content"]
# 평가 로직
reward = evaluate_custom_criteria(content)
rewards.append(reward)
return rewards


# 등록
REWARD_FUNCS_REGISTRY["custom"] = custom_reward
```

보상 함수 시스템은 Open R1의 핵심 구성 요소로, 다양한 평가 기준을 통해 모델의 추론 능력을 종합적으로 향상시킵니다. 각 보상 함수는 특정 측면에 특화되어 있으며, 이들의 조합을 통해 균형 잡힌 모델 훈련이 가능합니다.